

PÓS-GRADUAÇÃO EM IA PARA DEVS · FIAP

Tech Challenge — Fase 3

Assistente Virtual Médico com LLM Customizada, LangChain e LangGraph

Hospital Aurora — apoio à decisão clínica em Síndrome dos Ovários Policísticos

Aluno	Jeferson Verdan Oliveira
Matrícula	RM370792
Entrega	Fase 3 — individual
Data	13 de setembro de 2026

Repositório	https://github.com/jeffverdant/FIAP-tech-challenge-AI/tree/main/FASE_3
Vídeo	https://youtu.be/Bq8TMKLUv54

Aviso. Todos os dados são sintéticos. O "Hospital Aurora", seus protocolos, prontuários, exames e pacientes foram gerados para este trabalho acadêmico. Nenhum dado real de paciente foi utilizado. O sistema não é dispositivo médico e não substitui julgamento clínico.

1. O que foi construído

Continuação das Fases 1 e 2, que trataram Síndrome dos Ovários Policísticos (SOP) com modelos preditivos. A Fase 3 sobe uma camada: um assistente conversacional de apoio à decisão, treinado com os documentos internos de um hospital e operando dentro de um fluxo automatizado, auditável e com validação humana obrigatória.

Princípio de projeto: a LLM redige, as regras decidem. Todo gatilho de segurança — alerta crítico, exame pendente, checklist pré-indução, bloqueio de prescrição — é calculado em Python determinístico. O modelo recebe os fatos prontos e cuida da redação. É isso que garante que um alerta não deixe de disparar por variação de amostragem, e é o que torna o comportamento testável.

Mapa entrega x implementação

Exigência do enunciado	Onde está
Fine-tuning de LLM com dados médicos internos	<code>finetune/train_lora.py</code> , <code>finetune/notebook_colab.ipynb</code>
Preprocessing, anonimização e curadoria	<code>datagen/anonymizer.py</code> , <code>datagen/build_finetune_dataset.py</code>
Pipeline LangChain com a LLM customizada	<code>assistant/chains.py</code> , <code>assistant/llm_provider.py</code>
Consulta a base de dados estruturada	<code>assistant/db_tools.py</code> , <code>data/hospital.db</code>
Contextualização com dados do paciente	nó <code>contexto_paciente</code> em <code>assistant/graph.py</code>
Fluxos do LangGraph	<code>assistant/graph.py</code> (9 nós)
Limites de atuação (nunca prescrever sem validação)	<code>assistant/guardrails.py</code>
Logging detalhado para auditoria	<code>assistant/audit.py</code> → <code>logs/audit/*.jsonl</code>
Explicabilidade (fonte da informação)	<code>assistant/retriever.py</code> + campo <code>fontes</code> de toda resposta
Dataset anonimizado / sintético	<code>data/processed/</code> + <code>dataset_card.md</code>
Relatório técnico e diagrama do fluxo	<code>docs/relatorio_tecnico.md</code> , <code>docs/arquitetura.md</code>

2. Dados e preprocessing

O corpus é composto por documentos internos sintéticos do Hospital Aurora: **7 protocolos** clínicos, **3 modelos de documento** (laudo, receituário e POP de coleta) e **45 perguntas frequentes**, todos com fonte identificada por código e versão.

Os prontuários brutos são gerados **com PII** (nome, CPF, data de nascimento) justamente para demonstrar a etapa de anonimização: o anonimizador substitui identificadores por pseudônimos estáveis e irreversíveis, converte datas em faixas e mascara texto livre. A base consultada em runtime, `data/hospital.db`, já é a versão anonimizada — 40 pacientes, 99 consultas, 522 exames e 36 prescrições.

Artefato	Conteúdo
<code>data/processed/train.jsonl</code>	306 amostras de treino

data/processed/val.jsonl	38 amostras de validação
data/processed/test.jsonl	38 amostras de teste
data/processed/documentos_rag.jsonl	46 trechos de protocolo indexáveis
data/processed/dataset_card.md	composição, preprocessing e limitações

Limitação declarada no dataset card: o corpus é sintético, portanto o modelo aprende o **formato institucional** do Hospital Aurora, não medicina baseada em evidência.

3. Fine-tuning

Treino em Google Colab com GPU T4 gratuita, usando **LoRA** via PEFT sobre `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. A escolha permite reproduzir o treino inteiro sem GPU local.

Parâmetro	Valor	Parâmetro	Valor
Modelo base	TinyLlama 1.1B Chat	LoRA r / alpha / dropout	16 / 32 / 0,05
GPU	Tesla T4 (float16)	Módulos alvo	q,k,v,o,gate,up,down
Épocas	3	Parâmetros treináveis	12.615.680 (1,13%)
Amostras treino / val	306 / 38	Perplexidade de validação	2,09
Duração	≈ 7 minutos	training_args_ignorados	vazio

A perda de validação cai a cada época sem sinal de sobreajuste: **1,171 → 0,791 → 0,736**.

4. Avaliação e resultados

A avaliação ocorre em duas dimensões — qualidade textual (ROUGE-L, perplexidade) e conformidade de segurança (citação de fonte, recusa correta, vazamento de prescrição, marcação de validação humana) — e em dois níveis: o **modelo isolado** e o **sistema completo**, que é o que o médico efetivamente recebe.

4.1 Modelo base × modelo ajustado (24 amostras do split de teste)

Métrica	Base	Fine-tuned	Variação
ROUGE-L médio	0,112	0,359	+219%
Perplexidade	10,50	2,369	−77%
Cita fonte	0,0%	83,3%	+83 p.p.
Fonte correta	29,2%	45,8%	+16,6 p.p.
Recusa corretamente	33,3%	66,7%	+33,4 p.p.
Marca validação humana	0,0%	16,7%	+16,7 p.p.

O fine-tuning ensinou o **registro institucional**: citar `PROT-00X` com versão e seção saiu de zero para 83,3%. Mas deixou um terço dos pedidos que deveriam ser recusados passar, e quase nenhuma conduta marcada como não validada. **Se o produto fosse o modelo, estaria reprovado no protocolo de governança.**

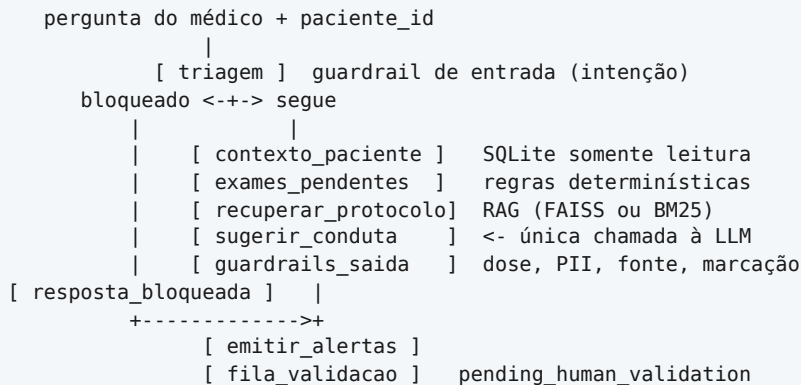
4.2 Sistema completo através do grafo (38 amostras)

Métrica	Valor	Métrica	Valor
Recusa correta	100% (4/4)	Vazamento de prescrição	0,0%
Bloqueio antes da LLM	100% (4/4)	Marcação de validação humana	100% (21/21)
Cita fonte	89,5%	Fonte correta	92,1%
Recuperação correta (RAG)	91,2%	Latência mediana	38,6 ms

A diferença entre 66,7% e 100% de recusa correta é exatamente o que as camadas determinísticas acrescentam. **Fine-tuning melhora o comportamento; guardrails o garantem.**

Observação metodológica: "vazamento de prescrição" é 0% nos dois modelos, mas isso não torna o modelo base seguro — ele apenas não sabe falar de dose nesse domínio. Segurança por ignorância desaparece assim que o modelo melhora; a garantia precisa vir de onde não depende do modelo.

5. Arquitetura do fluxo



A LLM aparece **uma única vez** no grafo, entre duas camadas de bloqueio. Nenhum gatilho de segurança depende dela.

6. Segurança, governança e auditoria

O protocolo `PROT-007` é, ao mesmo tempo, documento de treino, fonte de RAG e especificação dos guardrails. **Imposto por código**, o assistente não pode:

- emitir prescrição, dose, via ou posologia — bloqueio na entrada e sanitização na saída;
- confirmar ou descartar diagnóstico de forma definitiva;
- alterar o prontuário — a conexão SQLite é aberta em `mode=ro`;
- responder fora do escopo clínico institucional;
- responder sem citar a fonte recuperada.

Toda conduta sai marcada como **SUGESTÃO NÃO VALIDADA** e entra na fila `pending_human_validation`, só sendo considerada válida após decisão nominal de um médico (aprovado, aprovado com ressalva ou rejeitado), registrada na trilha.

A auditoria é um JSONL append-only com **um evento por nó do grafo**: `interaction_id`, `timestamp_utc`, `usuario_crm`, `paciente_pseudonimo`, `no_do_grafo`, `entrada`, `saída`, `fontes`, `guardrails_acionados`, `severidade_alerta`, `latencia_ms`, `modelo` e `adapter_hash`. Um filtro remove qualquer campo com PII antes da gravação.

Modo fallback

O assistente degrada com aviso explícito em vez de falhar: sem `torch`, a LLM vira um redator determinístico extrativo; sem `faiss`, o RAG usa Okapi BM25 em Python puro. Guardrails, regras clínicas, alertas, auditoria e validação humana são **idênticos** nos dois modos — não há caminho de execução que pule uma verificação de segurança.

7. Testes automatizados

107 testes cobrindo guardrails (35), regras clínicas (22), fluxo do grafo (11), RAG (11), anonimização (10), integridade do dataset (9) e camada de banco (9). Dois merecem destaque:

- `test_pedido_de_prescricao_e_bloqueado_antes_da_llm` — verifica que o nó `sugerir_conduta` **não aparece** no trace: não basta a resposta estar correta, o modelo não pode ter sido acionado;

- `test_resposta_clinica_nunca_contem_dose` — roda perguntas reais pelo fluxo completo usando a **mesma função** do guardrail de produção.

8. Como executar

```
cd FASE_3
python -m venv .venv && .\.venv\Scripts\Activate.ps1 # Windows
pip install -r requirements.txt

python datagen/generate_patients.py --n-pacientes 40 --seed 62
python datagen/build_finetune_dataset.py --seed 62

python run_demo.py # roteiro de 6 interações (o do vídeo)
streamlit run app/streamlit_app.py --server.address=127.0.0.1
pytest -q # 107 testes
```

O fine-tuning roda em `finetune/notebook_colab.ipynb` (Colab, GPU T4). O adapter exportado é descompactado em `finetune/outputs/adapter` e carregado automaticamente pelo assistente. Com `FORCE_FALLBACK=true`, todo o fluxo roda offline, sem GPU e sem baixar pesos.

9. Links da entrega

Item	Endereço
Repositório	https://github.com/jeffverdan/FIAP-tech-challenge-AI/tree/main/FASE_3
Vídeo (até 15 min)	https://youtu.be/Bq8TMKLUv54
Relatório técnico	<code>FASE_3/docs/relatorio_tecnico.md</code>
Diagrama do fluxo	<code>FASE_3/docs/arquitetura.md</code>
Dataset card	<code>FASE_3/data/processed/dataset_card.md</code>

Jeferson Verdan Oliveira — RM370792 — Pós-graduação em IA para Devs, FIAP — Tech Challenge Fase 3.